ASEGURAMIENTO DE CALIDAD DE SOFTWARE

Plan de Pruebas de API

Banco ERP — Public API de integración ACH (transferencias entrantes)

Dato	Detalle
API bajo prueba	BancoERP.PublicAPI v1 (ASP.NET Core, .NET 10)
Documento base	Guía resumida de integración ACH — Public API (Guia_BancoNexus.pdf)
Ambiente de ejecución	Local (docker compose): http://localhost:5254
Responsable	Equipo de QA — Luis Contreras
Fecha de ejecución	3 de octubre de 2026
Versión del documento	1.0

Contenido

0. Datos para iniciar

A. Analizar

Descripción de la API · Alcance · Suposiciones y dependencias · Riesgos y prioridades

B. Diseñar

Estrategia · Criterios de aceptación · Matriz de cobertura · Diseño de casos de prueba

C. Preparar

Ambiente · Datos de prueba · Herramientas · Trazabilidad

D. Ejecutar

Ejecución y evidencias · Resultados por caso

E. Reportar

Resumen ejecutivo · Incidencias · Conclusiones · Lecciones aprendidas

Anexos

Archivos entregados · Cambios aplicados al código · Log de la regresión final

Flujo seguido. El plan sigue los cinco pasos solicitados: **analizar** la API y sus riesgos, **diseñar** la estrategia y los casos, **preparar** el ambiente y los datos, **ejecutar** en ciclos con evidencia y **reportar** resultados, defectos y mejoras. Los defectos encontrados se corrigieron en el código y se verificaron con nuevos ciclos de regresión.

0. Datos para iniciar

Pregunta	Respuesta
¿Qué API se usará?	Banco ERP Public API: API REST que permite a un banco externo autenticarse con un usuario técnico, validar una cuenta destino y enviar transferencias ACH entrantes que se acreditan en Banco ERP. URL pública según la guía: http://157.137.186.211:5254. Las pruebas se ejecutaron en una instancia local equivalente (http://localhost:5254).
¿Cuáles endpoints tiene?	GET /api/Home · POST /api/external/auth/login · GET /api/external/ach/cuentas/search/{numeroCuenta} · POST /api/external/ach/transferencias
¿Requiere autenticación?	Sí. JWT Bearer (HS256) obtenido en el login, con vigencia de 1 hora. En cada solicitud se valida además que la sesión siga activa en la base de datos. La autorización exige el rol BANCO_EXTERNO y los permisos ach.public.auth.login, ach.public.cuentas.consultar y ach.public.transferencias.crear. /api/Home y el login son anónimos.
¿Qué métodos soporta?	Solo GET y POST. No hay PUT ni DELETE (un DELETE responde 405).
¿Qué respuestas se esperan?	JSON (application/json). Formato estándar {status, message, data, errors}; el login exitoso devuelve {accessToken, expiresAt, tokenType}. Códigos: 200, 201, 400, 401, 403, 404, 409, 422, 423 y 500 (guía §8), más 405 y 415 del framework.

A. Analizar

A.1 Descripción de la API

Propósito. La Public API es el punto de entrada de bancos terceros hacia Banco ERP para transferencias ACH entrantes. El flujo obligatorio de la guía tiene tres pasos: (1) autenticarse con el usuario técnico, (2) consultar la cuenta destino y confirmar que existe y está activa, y (3) enviar la transferencia con un codigo_confirmacion único que funciona como llave de idempotencia. Una transferencia válida acredita el saldo y registra la transferencia, un movimiento ACH_IN, el asiento contable y la auditoría.

Método	Ruta	Auth	Descripción
GET	/api/Home	No	Verificación básica de disponibilidad.
POST	/api/external/auth/login	No	Autentica al banco externo y devuelve un JWT.
GET	/api/external/ach/cuentas/search/{numeroCuenta}	Bearer	Valida que la cuenta destino exista y pueda recibir ACH.
POST	/api/external/ach/transferencias	Bearer	Registra y acredita una transferencia ACH entrante.

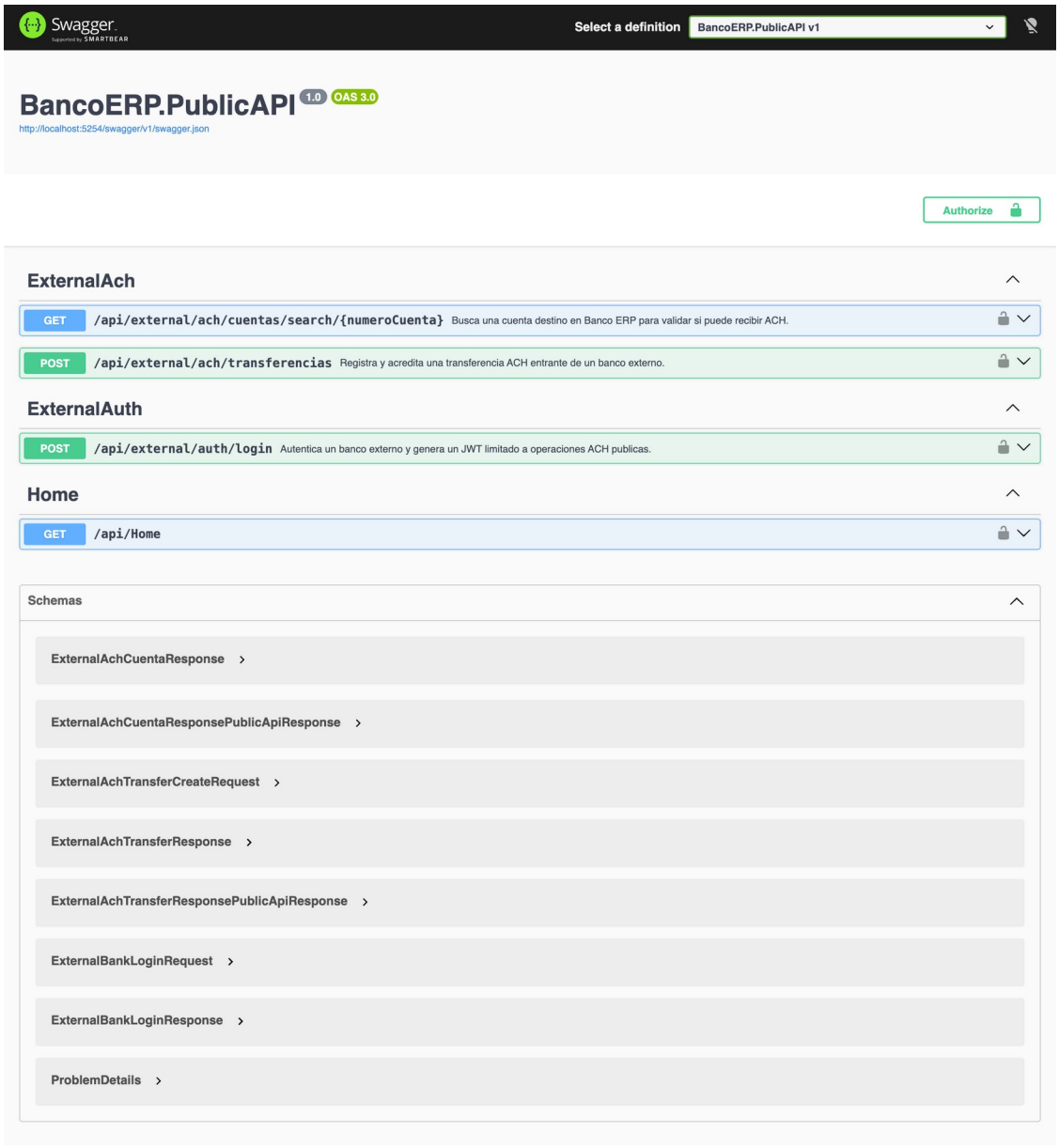


Figura 1. Swagger UI de la Public API con los cuatro endpoints y sus esquemas.

Entradas y salidas principales

Login: request y response 200 (contraseña y token enmascarados):

```
POST /api/external/auth/login
Content-Type: application/json


{
  "email": "bancoexterno@nexus.com",
  "password": "*******"
}
```

```
{
  "accessToken": "eyJhbGciOiJIUzI1...(JWT 1083 chars)",
  "expiresAt": "2026-10-04T00:33:57.7550854Z",
  "tokenType": "Bearer"
}
```


Consulta de cuenta: response 200 (TC-11):


```
GET /api/external/ach/cuentas/search/MO260 803 936 604
Authorization: Bearer {token}


{
  "status": true,
  "message": "Cuenta encontrada.",
  "data": {
    "numeroCuenta": "MO260 803 936 604",
    "nombreTitular": "Ana Sofia Garcia Mendez",
    "moneda": "Q",
    "estado": "ACTIVA",
    "puedeRecibirTransferencias": true
  },
  "errors": null
}
```


Transferencia: request y response 201 (TC-19):


```
POST /api/external/ach/transferencias
Authorization: Bearer {token}
Content-Type: application/json


{
  "codigo_confirmacion": "TRX-QA-2 026 100 323 3357-0001",
  "banco_externo": "DERBANCKS",
  "cuenta_externa": "1 234 567 890",
  "numeroCuentaDestino": "MO260 803 936 604",
  "monto": 500,
  "moneda": "Q",
  "referencia": "Transferencia ACH entrante",
  "tipo": "entrante"
}
```



```
{
  "status": true,
  "message": "Transferencia recibida correctamente.",
  "data": {
    "codigoConfirmacion": "TRX-QA-2 026 100 323 3357-0001",
    "numeroCuentaDestino": "MO260 803 936 604",
    "monto": 500,
    "moneda": "Q",
    "estado": "APROBADA",
    "fechaProcesamiento": "2026-10-03T23:33:58.549155Z"
  },
  "errors": null
}
```


Respuestas de error: duplicado 409 (TC-20) y validación 400 (TC-21):

```
{
  "status": false,
  "message": "La transferencia ya fue procesada.",
  "data": null,
  "errors": {
    "codigo_confirmacion": [
      "El codigo de confirmacion ya existe."
    ]
  }
}
```



```
{
  "status": false,
  "message": "Solicitud invalida.",
  "data": null,
  "errors": {
    "codigo_confirmacion": [
      "El codigo de confirmacion es requerido."
    ],
    "banco_externo": [
      "El banco externo es requerido."
    ],
    "cuenta_externa": [
      "La cuenta externa es requerida."
    ],
    "numeroCuentaDestino": [
      "El numero de cuenta destino es requerido."
    ],
    "monto": [
      "El monto debe ser mayor a cero."
    ],
    "moneda": [
      "La moneda soportada para ACH entrante es GTQ/Q."
    ]
  }
}
```


Campo del request	Regla (guía §6 y código)
codigo_confirmacion	Obligatorio y único; llave de idempotencia.
banco_externo	Obligatorio; identifica al banco que envía.
cuenta_externa	Obligatorio; cuenta origen en el banco externo.
numeroCuentaDestino	Obligatorio; cuenta existente y ACTIVA en Banco ERP.
monto	Obligatorio; mayor a 0 (acepta número o texto numérico).
moneda	Obligatorio; Q o GTQ.
referencia	Opcional; texto libre.
tipo	Opcional; si se envía debe ser "entrante".

A.2 Alcance de pruebas

Dentro del alcance:

- Los 4 endpoints de la Public API, en el flujo completo login → consulta → transferencia.
- Comportamientos funcionales: emisión del token, consulta de cuenta, acreditación y efectos en la base de datos (saldo, movimiento ACH_IN, asientos contables y auditoría).
- Validaciones de entrada: campos obligatorios, monto, moneda, tipo, formato JSON y Content-Type.
- Seguridad básica: autenticación (sin token, token alterado, sesión revocada), autorización (usuario inactivo, sin rol), bloqueo por intentos fallidos e intentos de inyección SQL.

- Manejo de errores y códigos HTTP contra la tabla de la guía (§8).
- Idempotencia del codigo_confirmacion, secuencial y concurrente.
- Rendimiento básico de la consulta de cuenta (tiempo de respuesta).

Fuera del alcance (explícito):

- El servidor público 157.137.186.211:5254: se excluye para no generar transacciones financieras reales ni bloquear al usuario técnico compartido con otros bancos.
- Pruebas de carga y estrés, rate limiting y expiración natural del token (1 h).
- Transporte seguro (HTTPS/TLS), CORS y configuración de infraestructura.
- La API interna (BancoERP.InternalAPI), el frontend y el módulo ACH saliente.
- Combinaciones finas de permisos (usuario BANCO_EXTERNO al que le falte solo uno de los tres permisos).

A.3 Suposiciones y dependencias

Aspecto	Situación
Autenticación	JWT Bearer firmado con secreto simétrico (Jwt__SecretKey ≥ 32 caracteres), con issuer y audience validados. Cada token incluye el id de sesión (sid), que se valida contra seguridad.sesiones en cada solicitud.
Base de datos	PostgreSQL 16. La transferencia se ejecuta en el procedimiento ach.sp_registrar_transferencia_entrante_publica (013_external_public_ach.sql), que bloquea la cuenta (FOR UPDATE) y usa pg_advisory_xact_lock por código.
Entornos	Local con docker compose (servicios postgres y public-api), en modo Development con dotnet watch. Existe un despliegue público según la guía, que no se usó. No hay ambiente de staging.
Datos	Seeds del repositorio (cuentas y usuario técnico) más el script 01_preparar_datos_qa.sql, que crea los usuarios QA.
Suposición	La instancia local con los scripts de deploy/production replica el esquema de producción. Por eso los defectos de base de datos que se encuentren aplican también al servidor público.
Dependencia externa	Ninguna: el modelo es request/response, sin webhooks ni callbacks (guía §9).

A.4 Riesgos y prioridades

ID	Riesgo	Prioridad	Justificación
R-01	Transferencias válidas no acreditadas o acreditadas con datos incorrectos.	Alta	Impacto financiero directo; es la función principal de la API.
R-02	Doble acreditación por reintentos o envíos concurrentes del mismo código.	Alta	Pérdida de dinero; la guía exige idempotencia por codigo_confirmacion.
R-03	Acceso no autorizado: sin token, token alterado, sesión revocada o usuario sin rol.	Alta	Expone datos de clientes y permite mover fondos.
R-04	Ataques de fuerza bruta sobre el usuario técnico.	Media	Hay mitigación por bloqueo, pero un bloqueo indebido detiene la integración.
R-05	Validación insuficiente de entradas (montos, moneda, tipo, formato).	Alta	Montos inválidos o redondeados alteran saldos.
R-06	Fuga de información interna en mensajes de error.	Media	Facilita ataques (CWE-209); la respuesta llega a un tercero.
R-07	Códigos HTTP o formatos distintos a los documentados en la guía.	Media	El banco externo decide reintentos según el código; un código erróneo rompe su integración.

ID	Riesgo	Prioridad	Justificación
R-08	Disponibilidad o tiempo de respuesta degradados.	Baja	Volumen esperado bajo; se valida solo una línea base.

B. Diseñar

B.1 Estrategia de prueba

Tipo de prueba	Qué se prueba	Casos
Funcionales (positivas)	Login, consulta de cuenta y transferencia válidas, con verificación de efectos en BD.	TC-01, 02, 11, 12, 19, 29
Negativas / validación	Campos vacíos, montos inválidos, moneda y tipo no soportados, cuentas inexistentes o bloqueadas.	TC-05, 13, 14, 21–27, 30
Seguridad (autenticación/autorización)	Credenciales erróneas, usuario inactivo o sin rol, bloqueo, token ausente/alterado/revocado, inyección SQL.	TC-03, 04, 07–10, 15–18, 28
Manejo de errores / estados HTTP	JSON malformado, método y Content-Type no soportados, desbordamiento numérico.	TC-06, 31, 33, 34
Regla de negocio / concurrencia	Idempotencia secuencial y concurrente.	TC-20, 32
Rendimiento básico	Tiempo de respuesta de la consulta (p95).	TC-35

Herramientas y enfoque:

- **Swagger UI** (/swagger): exploración manual y evidencias visuales. Las capturas se automatizaron con **Playwright + Google Chrome** (capturas_swagger.mjs), enmascarando contraseña y JWT.
- **Runner automatizado en Node.js** (run_api_tests.mjs, fetch nativo): ejecuta los 35 casos, valida código HTTP, estructura y mensajes, y guarda request/response, tiempos y verificaciones en JSON y log.
- **Verificación de caja gris en PostgreSQL** (psql en el contenedor): saldo, movimiento ACH_IN, asientos, auditoría y estado de usuarios y sesiones.
- **xUnit** (BancoERP.UnitTests): pruebas unitarias de regresión para las validaciones corregidas.
- **Ciclos**: línea base sobre el código original → corrección de defectos → re-test y regresión completa hasta estabilizar.

B.2 Criterios de aceptación

ID	Funcionalidad	Criterio de aceptación
CA-01	Disponibilidad	GET /api/Home devuelve 200 con estado "Conectado" y fecha.
CA-02	Login exitoso	Devuelve 200 con {accessToken: JWT de 3 partes, expiresAt futuro ≤ 60 min, tokenType: "Bearer"}.
CA-03	Login rechazado	Credenciales inválidas → 401 con mensaje genérico y sin token; campos vacíos → 400; usuario inactivo o sin rol BANCO_EXTERNO → 403; tras 5 intentos fallidos → bloqueo y 423.
CA-04	Consulta exitosa	Devuelve 200 con {status:true, message:"Cuenta encontrada.", data:{numeroCuenta, nombreTitular, moneda:"Q", estado:"ACTIVA", puedeRecibirTransferencias:true}}; la búsqueda no distingue mayúsculas.
CA-05	Consulta rechazada	Cuenta inexistente → 404; cuenta no ACTIVA → 422, sin datos del titular.
CA-06	Transferencia exitosa	Devuelve 201 con data.estado "APROBADA" y moneda "Q". En BD: saldo + monto, movimiento ACH_IN, asiento de partida doble y evento de auditoría.
CA-07	Idempotencia	Un codigo_confirmacion repetido (secuencial o concurrente) → 409 con errors.codigo_confirmacion; se acredita una sola vez.
CA-08	Validación de request	Faltan obligatorios, monto ≤ 0, monto con más de 2 decimales o fuera de rango, o tipo ≠ entrante → 400 con errors por campo y sin afectar el saldo.

ID	Funcionalidad	Criterio de aceptación
CA-09	Reglas de negocio	Cuenta destino inexistente → 404; cuenta no apta o moneda inválida → 422 (guía §8).
CA-10	Protección de endpoints	Sin token, con token alterado o con sesión revocada → 401, sin datos.
CA-11	Manejo de errores	Errores de formato, método o media type → 400/405/415, sin stack traces ni mensajes internos de la BD.
CA-12	Rendimiento básico	p95 < 500 ms en la consulta de cuenta (local, 20 solicitudes secuenciales).

B.3 Matriz de cobertura

Endpoint	Funcionalidad	Tipo de prueba	CA	Casos
GET /api/Home	Disponibilidad	Funcional	CA-01	TC-01
POST /auth/login	Autenticación exitosa	Funcional	CA-02	TC-02
POST /auth/login	Rechazo de credenciales y validación	Negativa / Validación	CA-03	TC-03, 04, 05
POST /auth/login	Autorización por estado y rol	Seguridad	CA-03	TC-07, 08
POST /auth/login	Bloqueo e inyección SQL	Seguridad	CA-03	TC-09, 10
POST /auth/login	JSON malformado	Manejo de errores	CA-11	TC-06
GET /cuentas/search	Consulta válida	Funcional	CA-04	TC-11, 12
GET /cuentas/search	Cuenta inexistente o no apta	Negativa / Seguridad	CA-05	TC-13, 14, 18
GET /cuentas/search	Autenticación del endpoint	Seguridad	CA-10	TC-15, 16, 17
GET /cuentas/search	Tiempo de respuesta	Rendimiento	CA-12	TC-35
POST /transferencias	Acreditación	Funcional	CA-06	TC-19, 29
POST /transferencias	Idempotencia	Regla de negocio / Concurrencia	CA-07	TC-20, 32
POST /transferencias	Validación de request	Negativa / Validación	CA-08	TC-21, 22, 23, 25, 30
POST /transferencias	Reglas de negocio	Negativa	CA-09	TC-24, 26, 27
POST /transferencias	Autenticación del endpoint	Seguridad	CA-10	TC-28
POST /transferencias	Estados HTTP y errores	Manejo de errores	CA-11	TC-31, 33, 34

Las rutas abreviadas corresponden a /api/external/auth/login, /api/external/ach/cuentas/search/{numeroCuenta} y /api/external/ach/transferencias.

B.4 Diseño de casos de prueba

Cada caso indica el endpoint, las precondiciones, los datos de entrada, el resultado esperado, el tipo de prueba y la severidad si falla. **Pasos comunes** (salvo indicación en las precondiciones): (1) obtener un token con TC-02, (2) enviar la solicitud con `Authorization: Bearer {token}`, (3) verificar el código HTTP y la estructura de la respuesta y (4) verificar en la BD los efectos o la ausencia de efectos. Los códigos de confirmación siguen el patrón `TRX-QA-<runId>-NNNN` para que sean únicos en cada corrida.

ID	Endpoint + método	Precondiciones	Datos de entrada	Resultado esperado	Tipo	Sev.
TC-01	GET /api/Home Disponibilidad /api/Home	API levantada.	GET /api/Home sin token.	200; estado = "Conectado"; incluye fecha.	Funcional	Media
TC-02	POST /api/external/auth/login Login con usuario tecnico valido	Usuario técnico activo, rol BANCO_EXTERNO, permiso ach.public.auth.login.	{"email":"bancoexterno@nexus.com","password":"*******"}	200; accessToken JWT (3 partes); tokenType "Bearer"; expiresAt futuro ≤ 60 min.	Funcional	Alta
TC-03	POST /api/external/auth/login Login con contrasena incorrecta	Usuario QA de bloqueo con 0 intentos fallidos.	Email QA + contraseña incorrecta.	401; status=false; mensaje "Intento 1 de 5"; sin token.	Negativa / Seguridad	Alta
TC-04	POST /api/external/auth/login Login con usuario inexistente	Ninguna.	Email inexistente + cualquier contraseña.	401; mensaje genérico "Credenciales incorrectas." (no revela si el usuario existe).	Negativa / Seguridad	Media
TC-05	POST /api/external/auth/login Login con campos vacios	Ninguna.	{"email":"","password":""}	400; mensaje de campos requeridos.	Negativa / Validacion	Media
TC-06	POST /api/external/auth/login Login con JSON malformado	Ninguna.	Body JSON truncado (malformado).	400; sin stack trace ni nombres internos.	Manejo de errores	Baja
TC-07	POST /api/external/auth/login Login de usuario inactivo	Usuario QA inactivo con rol BANCO_EXTERNO.	Credenciales válidas del usuario inactivo.	403; mensaje de usuario inactivo.	Seguridad (autorizacion)	Alta
TC-08	POST /api/external/auth/login Login de usuario sin rol BANCO_EXTERNO	Usuario QA activo sin rol BANCO_EXTERNO.	Credenciales válidas del usuario sin rol.	403; "no esta autorizado"; sin token.	Seguridad (autorizacion)	Alta
TC-09	POST /api/external/auth/login Bloqueo tras 5 intentos fallidos	TC-03 ejecutado (1 intento fallido).	4 intentos fallidos más y luego la contraseña correcta.	Intentos 2-5 → 401 (el 5.º informa bloqueo); contraseña correcta → 423; BD: bloqueado = true con bloqueado_hasta.	Seguridad	Alta
TC-10	POST /api/external/auth/login Login con intento de inyeccion SQL	Ninguna.	{"email":"' OR '1'='1' --","password":"' OR '1'='1"}	401; sin token.	Seguridad	Alta
TC-11	GET /api/external/ach/cuentas/search/{numeroCuenta} Consulta de cuenta destino activa	Token válido; cuenta MO260803936604 ACTIVA.	GET .../search/MO260803936604	200; status=true; data con numeroCuenta, nombreTitular, moneda "Q", estado "ACTIVA", puedeRecibirTransferencias=true.	Funcional	Alta

ID	Endpoint + método	Precondiciones	Datos de entrada	Resultado esperado	Tipo	Sev.
TC-12	GET /api/external/ach/cuentas/search/{numeroCuenta} Consulta de cuenta en minusculas (normalizacion)	Token válido.	GET .../search/mo260803936604 (minúsculas)	200; numeroCuenta normalizado en mayúsculas.	Funcional	Baja
TC-13	GET /api/external/ach/cuentas/search/{numeroCuenta} Consulta de cuenta inexistente	Token válido.	GET .../search/MO-2026-99999	404; "Cuenta no encontrada."	Negativa	Media
TC-14	GET /api/external/ach/cuentas/search/{numeroCuenta} Consulta de cuenta BLOQUEADA	Token válido; cuenta AH260901838371 BLOQUEADA.	GET .../search/AH260901838371	422; "La cuenta no puede recibir transferencias."; sin datos del titular.	Negativa / Regla de negocio	Alta
TC-15	GET /api/external/ach/cuentas/search/{numeroCuenta} Consulta de cuenta sin token	Ninguna.	GET .../search/MO260803936604 sin Authorization.	401; no expone datos.	Seguridad (autenticacion)	Alta
TC-16	GET /api/external/ach/cuentas/search/{numeroCuenta} Consulta de cuenta con token alterado	Token válido.	Mismo token con la firma modificada.	401.	Seguridad (autenticacion)	Alta
TC-17	GET /api/external/ach/cuentas/search/{numeroCuenta} Consulta con token de sesion revocada	Login nuevo; sesión marcada como cerrada en seguridad.sesiones.	GET con el token de la sesión revocada.	401.	Seguridad (autenticacion)	Alta
TC-18	GET /api/external/ach/cuentas/search/{numeroCuenta} Consulta con intento de inyeccion SQL	Token válido.	GET .../search/X' OR '1'='1 (URL-encoded)	404; sin datos de otra cuenta.	Seguridad	Alta
TC-19	POST /api/external/ach/transferencias Transferencia ACH entrante valida	Token válido; cuenta ACTIVA; código único.	Body de la guía: monto 500.00, moneda "Q", tipo "entrante".	201; estado "APROBADA"; moneda "Q". BD: saldo +500, movimiento ACH_IN, asiento ≥ 2 líneas, evento de auditoría.	Funcional	Alta
TC-20	POST /api/external/ach/transferencias Transferencia duplicada (idempotencia)	TC-19 aprobado.	Mismo body y mismo codigo_confirmacion.	409; errors.codigo_confirmacion; saldo sin cambios.	Negativa / Regla de negocio	Alta

ID	Endpoint + método	Precondiciones	Datos de entrada	Resultado esperado	Tipo	Sev.
TC-21	POST /api/external/ach/transferencias Transferencia con body vacio	Token válido.	{}	400; errors con los 6 campos obligatorios.	Negativa / Validacion	Media
TC-22	POST /api/external/ach/transferencias Transferencia con monto 0	Token válido.	monto: 0	400; errors.monto.	Negativa / Validacion	Media
TC-23	POST /api/external/ach/transferencias Transferencia con monto negativo	Token válido.	monto: -100	400; saldo sin cambios.	Negativa / Validacion	Alta
TC-24	POST /api/external/ach/transferencias Transferencia con moneda no soportada (USD)	Token válido.	moneda: "USD"	422 (guía §8: moneda inválida); errors.moneda.	Negativa / Regla de negocio	Media
TC-25	POST /api/external/ach/transferencias Transferencia con tipo distinto de entrante	Token válido.	tipo: "saliente"	400; errors.tipo.	Negativa / Validacion	Media
TC-26	POST /api/external/ach/transferencias Transferencia a cuenta inexistente	Token válido.	numeroCuentaDestino: "MO-2026-99999"	404.	Negativa	Media
TC-27	POST /api/external/ach/transferencias Transferencia a cuenta BLOQUEADA	Token válido; cuenta BLOQUEADA.	numeroCuentaDestino: "AH260901838371"	422; saldo de la cuenta sin cambios.	Negativa / Regla de negocio	Alta
TC-28	POST /api/external/ach/transferencias Transferencia sin token	Ninguna.	Body válido sin Authorization.	401; saldo sin cambios.	Seguridad (autenticacion)	Alta
TC-29	POST /api/external/ach/transferencias Transferencia con monto como texto y moneda GTQ	Token válido; cuenta ACTIVA.	monto: "150.75" (texto), moneda: "GTQ"	201; monto 150.75; moneda normalizada a "Q".	Funcional	Media
TC-30	POST /api/external/ach/transferencias Transferencia con monto de 3 decimales	Token válido.	monto: 10.555	400; no se acredita un monto distinto al enviado.	Negativa / Validacion	Alta
TC-31	POST /api/external/ach/transferencias Transferencia con monto fuera de rango numerico	Token válido.	monto: 100000000000000000	400 controlado; sin mensajes internos de BD.	Manejo de errores	Media

ID	Endpoint + método	Precondiciones	Datos de entrada	Resultado esperado	Tipo	Sev.
TC-32	POST /api/external/ach/transferencias Envio concurrente del mismo codigo (5 solicitudes)	Token válido; código único.	5 POST simultáneos con el mismo codigo_confirmacion (monto 25).	Exactamente 1×201 y 4×409; saldo +25 una sola vez.	Regla de negocio / Concurrencia	Alta
TC-33	DELETE /api/external/ach/transferencias Metodo HTTP no soportado	Token válido.	DELETE /api/external/ach/transferencias	405.	Manejo de errores	Baja
TC-34	POST /api/external/ach/transferencias Content-Type no soportado	Token válido.	POST con Content-Type: text/plain	415.	Manejo de errores	Baja
TC-35	GET /api/external/ach/cuentas/search/{ numeroCuenta} Tiempo de respuesta de consulta de cuenta (20 llamadas)	Token válido; cuenta ACTIVA.	20 GET secuenciales a la misma cuenta.	p95 < 500 ms.	Rendimiento basico	Baja

Pasos específicos de casos compuestos

- **TC-09 (bloqueo):** enviar 4 contraseñas erróneas consecutivas al usuario QA de bloqueo (5 en total con TC-03), luego enviar la contraseña correcta y consultar seguridad.usuarios.bloqueado y bloqueado_hasta.
- **TC-17 (sesión revocada):** hacer login, leer el claim sid del JWT, actualizar seguridad.sesiones.estado = false para ese id y llamar la consulta con ese token.
- **TC-32 (concurrencia):** leer el saldo, enviar 5 solicitudes POST en paralelo (Promise.all) con el mismo body y código, y comparar los códigos obtenidos con la diferencia de saldo.

C. Preparar

C.1 Ambiente de prueba

Componente	Versión / valor	Notas
Entorno	Local (docker compose, proyecto bancoerp-dev)	ASPNETCORE_ENVIRONMENT=Development
Public API	BancoERP.PublicAPI, .NET 10, dotnet watch	Puerto 5254 → 8080 del contenedor
Base de datos	PostgreSQL 16-alpine	Puerto 5433; inicializada con deploy/production + seeds
Swagger UI	http://localhost:5254/swagger/index.html	Autorización Bearer integrada
Cliente de pruebas	Node.js 25.9 (fetch nativo)	run_api_tests.mjs
Navegador	Google Chrome + playwright-core	capturas_swagger.mjs (headless)
Pruebas unitarias	xUnit sobre .NET SDK 10.0	BancoERP.UnitTests

Variables de configuración (los secretos no se escriben en el documento ni en los scripts):

Variable	Uso	Origen
BASE_URL	URL base de la API	Por defecto http://localhost:5254
BANCO_EXTERNO_EMAIL	Usuario técnico	bancoexterno@nexus.com
BANCO_EXTERNO_PASSWORD	Contraseña del usuario técnico	Se lee de AchExternalAuth__Password en .env.local (no versionado)
QA_PASSWORD	Contraseña de los usuarios QA	Variable de entorno del ejecutor
PG_CONTAINER	Contenedor para verificaciones en BD	bancoerp-dev-postgres-1
Jwt__ExpirationHours / Security__MaxFailedAccessAttempts	Vigencia del token / intentos antes de bloqueo	1 hora / 5 intentos (.env.local)

Levantar el ambiente:

```
docker compose up -d --no-build postgres public-api
curl http://localhost:5254/api/Home      # 200 = API lista
```

C.2 Datos de prueba

Recurso	Valor	Estado / rol	Uso	Generación
Usuario técnico	bancoexterno@nexus.com	Activo, BANCO_EXTERNO	Flujo principal	seeds/ seguridad_admin_seed.sql
Usuario QA	qa.externo.bloqueo@test.com	Activo, BANCO_EXTERNO	TC-03, TC-09	01_preparar_datos_qa.sql
Usuario QA	qa.externo.inactivo@test.com	Inactivo, BANCO_EXTERNO	TC-07	01_preparar_datos_qa.sql
Usuario QA	qa.sin.rol@test.com	Activo, sin rol	TC-08	01_preparar_datos_qa.sql
Cuenta destino	MO260 803 936 604	ACTIVA, GTQ	Consulta y transferencias	seeds/ demo_data_seed.sql

Recurso	Valor	Estado / rol	Uso	Generación
Cuenta destino	AH260 730 742 944	ACTIVA, GTQ	TC-29 a TC-32	seeds/ demo_data_seed.sql
Cuenta destino	MO260 812 213 864	ACTIVA, GTQ	Capturas de Swagger	seeds/ demo_data_seed.sql
Cuenta destino	AH260 901 838 371	BLOQUEADA, GTQ	TC-14, TC-27	seeds/ demo_data_seed.sql
Cuenta inexistente	MO-2026-99 999	—	TC-13, TC-26	Valor inventado (formato de la guía)
Códigos de confirmación	TRX-QA-<runId>-NNNN	Únicos por corrida	Transferencias	Generados por el runner

Los usuarios QA se crearon con un hash bcrypt ($2y$, costo 10) generado con htpasswd -bnBC 10. El script es idempotente: antes de cada corrida restablece intentos fallidos y bloqueos, incluido el del usuario técnico. Se usaron usuarios QA aparte para no bloquear la credencial real del banco externo. **Nota:** la cuenta de ejemplo de la guía (MO-2026-00003) no existe en los datos de demostración, donde las cuentas tienen el formato MO/AH + 12 dígitos.

C.3 Preparación de herramientas

Ejecución automatizada. Los scripts están en ENTREGA_02_PLAN_PRUEBAS_API/scripts/:

```
ENTREGA_02_PLAN_PRUEBAS_API/
├── Plan_Pruebas_API_BancoERP.docx
├── scripts/
│   ├── 01_preparar_datos_qa.sql     # usuarios QA + reset de bloqueos (idempotente)
│   ├── run_api_tests.sh             # prepara datos y ejecuta la batería
│   ├── run_api_tests.mjs            # 35 casos + verificaciones en BD
│   └── capturas_swagger.mjs         # capturas de Swagger con Playwright
└── evidencias/
    ├── resultados/   cicloN_resultados.json, cicloN_ejecucion.log (N = 1..4)
    └── swagger/      01..13_*.png
```

```
# 1) Batería completa (lee la credencial del usuario técnico de .env.local)
QA_PASSWORD='***' ./ENTREGA_02_PLAN_PRUEBAS_API/scripts/run_api_tests.sh

# 2) Capturas de Swagger
npm i playwright-core
BANCO_EXTERNO_PASSWORD=*** node ENTREGA_02_PLAN_PRUEBAS_API/scripts/capturas_swagger.mjs

# 3) Pruebas unitarias
dotnet test Backend/BancoERP.UnitTests/BancoERP.UnitTests.csproj
```

Ejecución manual en Swagger. (1) Abrir /swagger, (2) ejecutar el login con *Try it out*, (3) pegar el accessToken en **Authorize** (esquema Bearer), (4) ejecutar la consulta y la transferencia con los datos de C.2. Cada transferencia necesita un codigo_confirmacion nuevo.

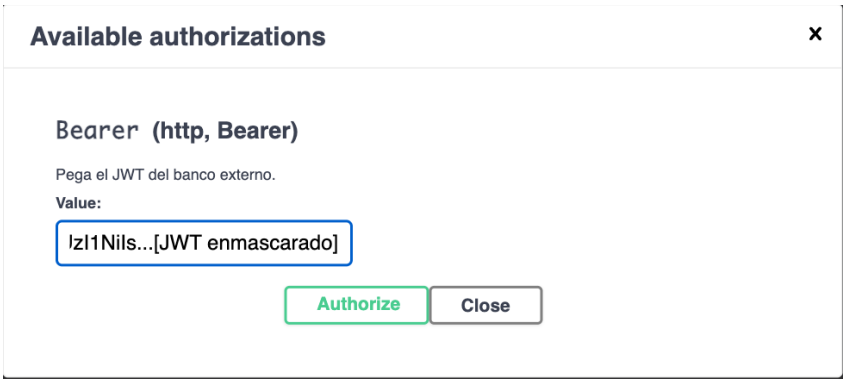


Figura 2. Diálogo Authorize de Swagger con el JWT del banco externo (enmascarado).

C.4 Trazabilidad

La trazabilidad se encadena así: **riesgo → criterio de aceptación → caso de prueba → resultado → defecto**. Cada riesgo de A.4 se cubre con al menos un criterio de B.2. Cada caso de B.4 declara su criterio y su riesgo. Los resultados de D.2 y los defectos de E.2 referencian los casos que los evidencian. Así se puede responder, por ejemplo, qué riesgo materializó un defecto o qué casos deben repetirse si cambia una regla.

Riesgo	Prioridad	Criterios	Casos	Defectos detectados
R-01 Acreditación	Alta	CA-04, CA-06, CA-09	TC-11, 14, 19, 27, 29	DEF-01, DEF-04, DEF-08
R-02 Doble acreditación	Alta	CA-07	TC-20, 32	— (enmascarado por DEF-01 en C1)
R-03 Acceso no autorizado	Alta	CA-02, CA-03, CA-10	TC-02–04, 07, 08, 10, 15–18, 28	Ninguno
R-04 Fuerza bruta	Media	CA-03	TC-03, 09	Ninguno
R-05 Validación de entradas	Alta	CA-08	TC-05, 12, 18, 21–23, 25, 30	DEF-05
R-06 Fuga de información	Media	CA-11	TC-06, 31	DEF-03, DEF-07
R-07 Contrato HTTP	Media	CA-05, CA-09, CA-11	TC-13, 24, 26, 33, 34	DEF-02, DEF-06
R-08 Disponibilidad / rendimiento	Baja	CA-01, CA-12	TC-01, 35	Ninguno

D. Ejecutar

D.1 Ejecución

Fecha: 3 de octubre de 2026 (horas en UTC-6). **Responsable:** equipo de QA (Luis Contreras). Se hicieron cuatro ciclos completos de los 35 casos con el runner automatizado, más una corrida manual en Swagger para las evidencias visuales.

Ciclo	Hora	Run ID	Código evaluado	Ejec.	Pass	Fail	Blocked
1	17:23	20261003232348	Línea base (código original)	35	26	9	0
2	17:26	20261003232621	Corrige DEF-01, 02, 03 y 05	35	30	5	0
3	17:27	20261003232711	Corrige DEF-04 (revelado en C2)	35	34	1	0
4	17:33	20261003233357	Corrige DEF-08; regresión final	35	34	1	0

Evidencia de ejecución: logs y reportes JSON de cada ciclo en evidencias/resultados/ (request, response, tiempo y resultado de cada verificación; contraseñas y JWT enmascarados), capturas de Swagger en evidencias/swagger/ y la salida de xUnit (16/16 pruebas aprobadas tras las correcciones). Extracto del log del ciclo 1, donde se detectó el defecto principal:

```
[FAIL] TC-19 Transferencia ACH entrante valida -> HTTP 422 (31 ms)
[FAIL] TC-26 Transferencia a cuenta inexistente -> HTTP 422 (7 ms)
[FAIL] TC-31 Transferencia con monto fuera de rango numerico -> HTTP 422 (8 ms)
       x no expone mensajes internos de base de datos | {"status":false,"message":"numeric field overflow","data":null,"errors":null}
Resumen: 35 ejecutados | 26 Pass | 9 Fail | 0 Blocked
```

Evidencias de Swagger

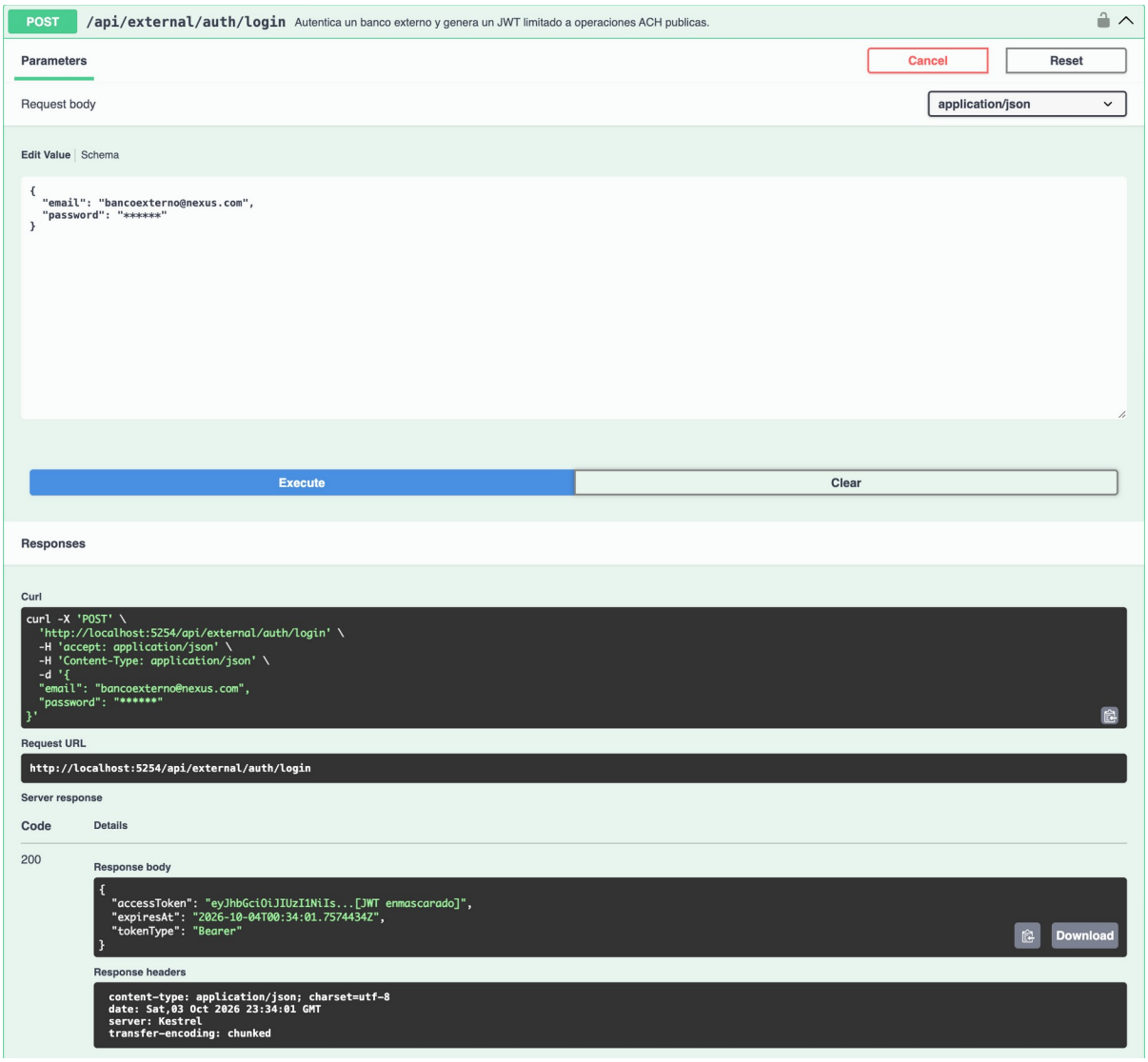


Figura 3. TC-02: login del usuario técnico, 200 con JWT (enmascarado).

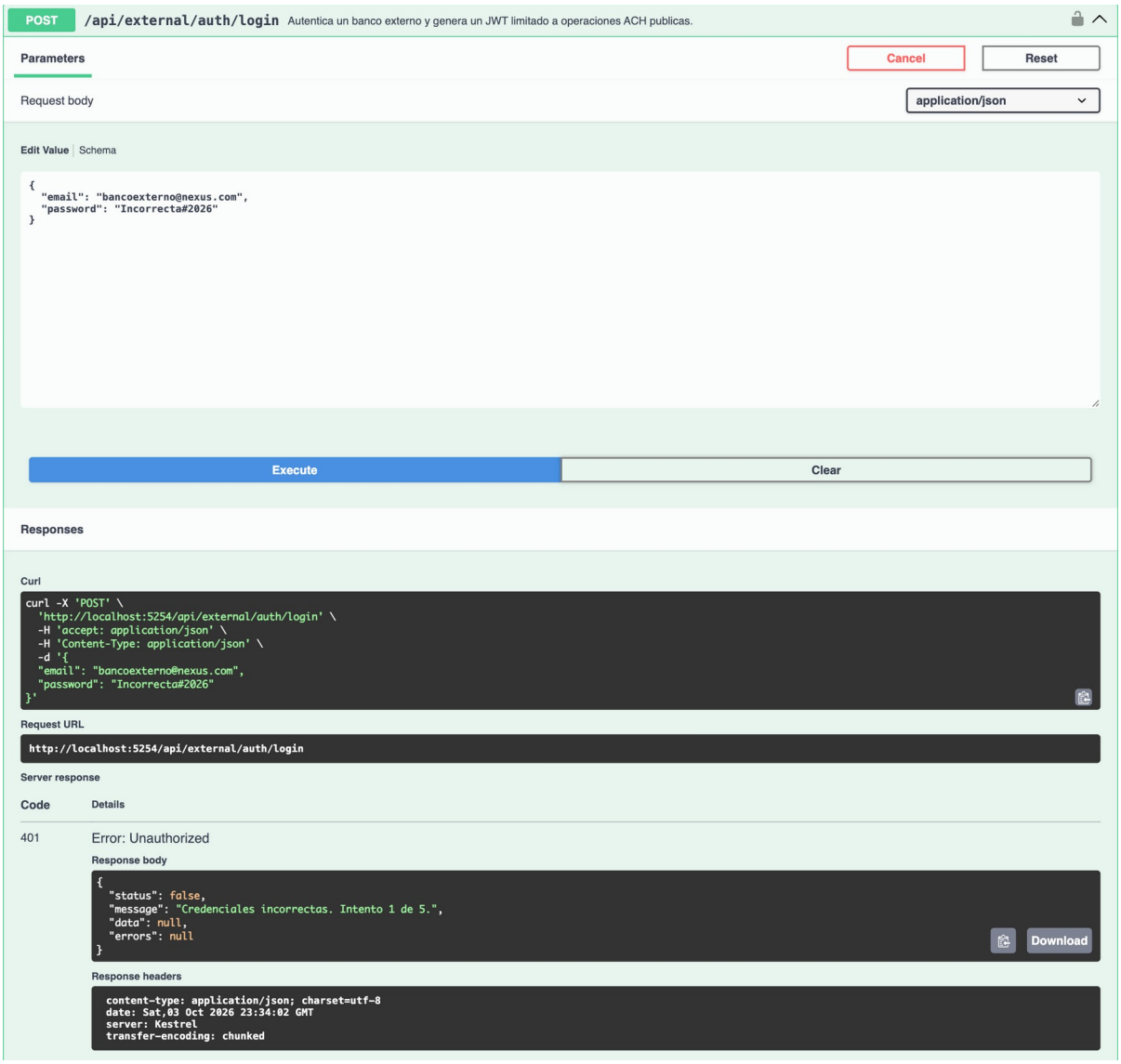


Figura 4. TC-03 (equivalente en Swagger): contraseña incorrecta, 401 con conteo de intentos.

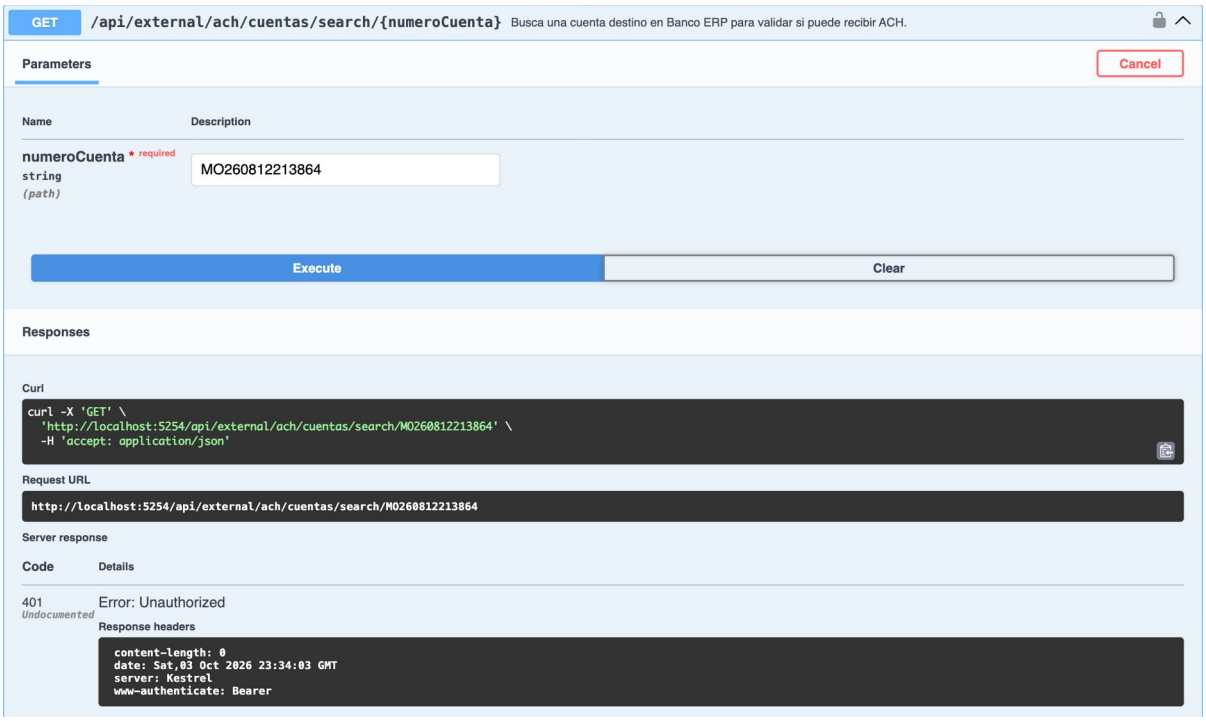


Figura 5. TC-15: consulta de cuenta sin token, 401.

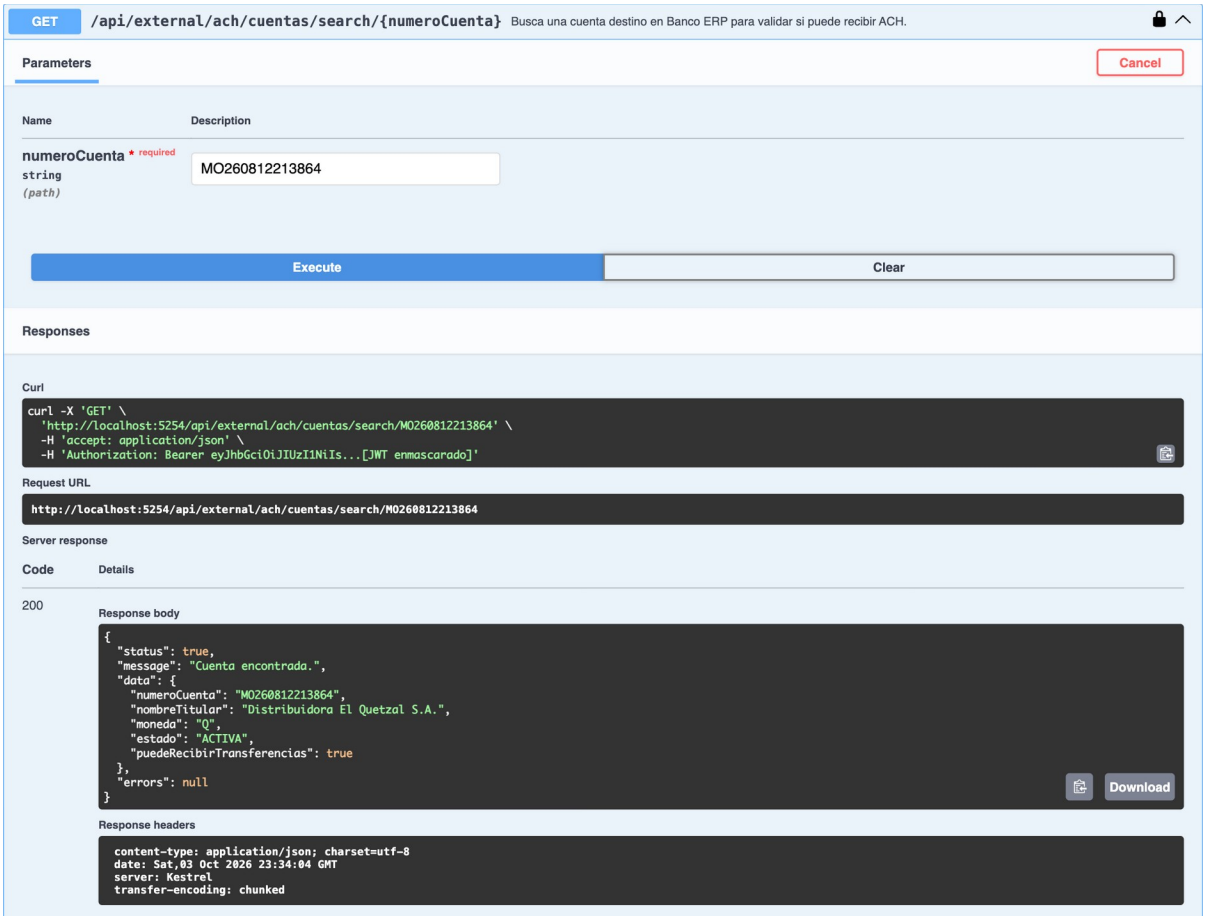


Figura 6. TC-11: consulta de cuenta activa, 200 con datos del titular.

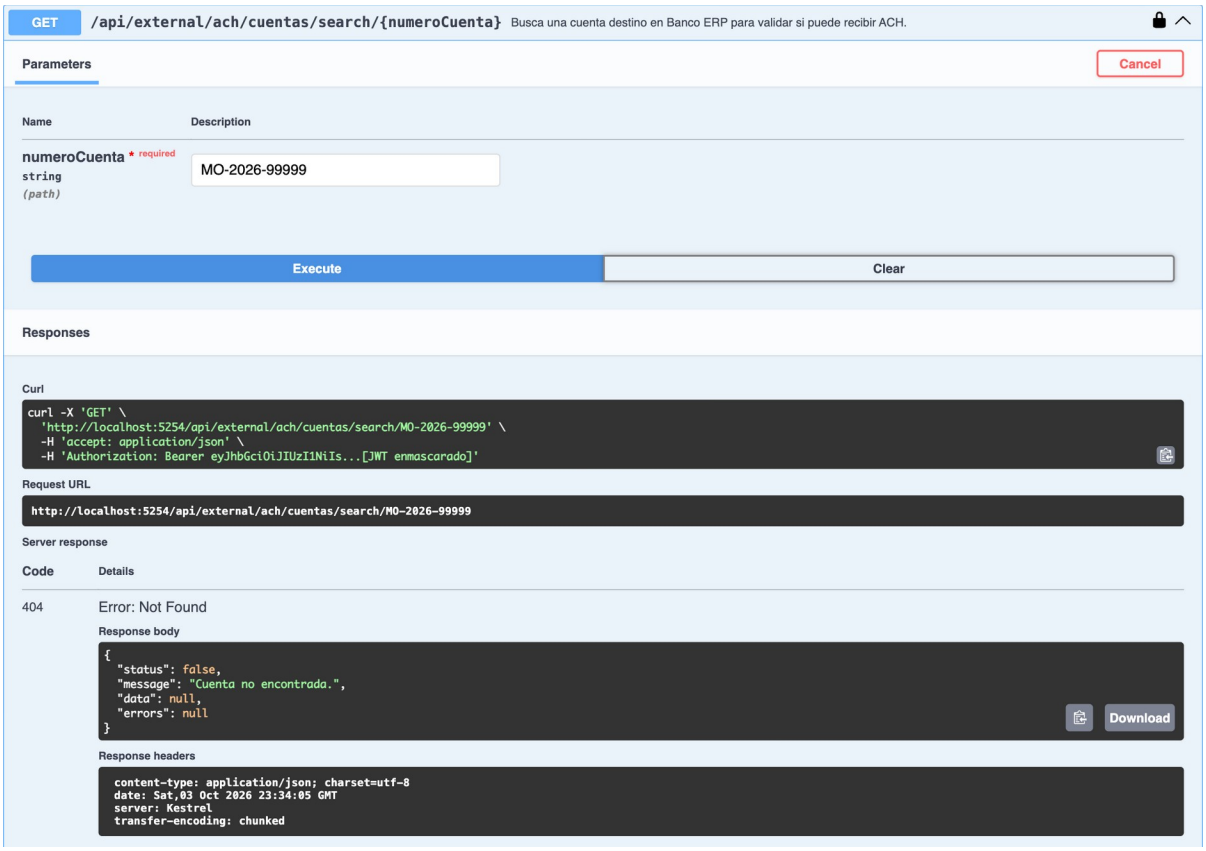


Figura 7. TC-13: cuenta inexistente, 404.

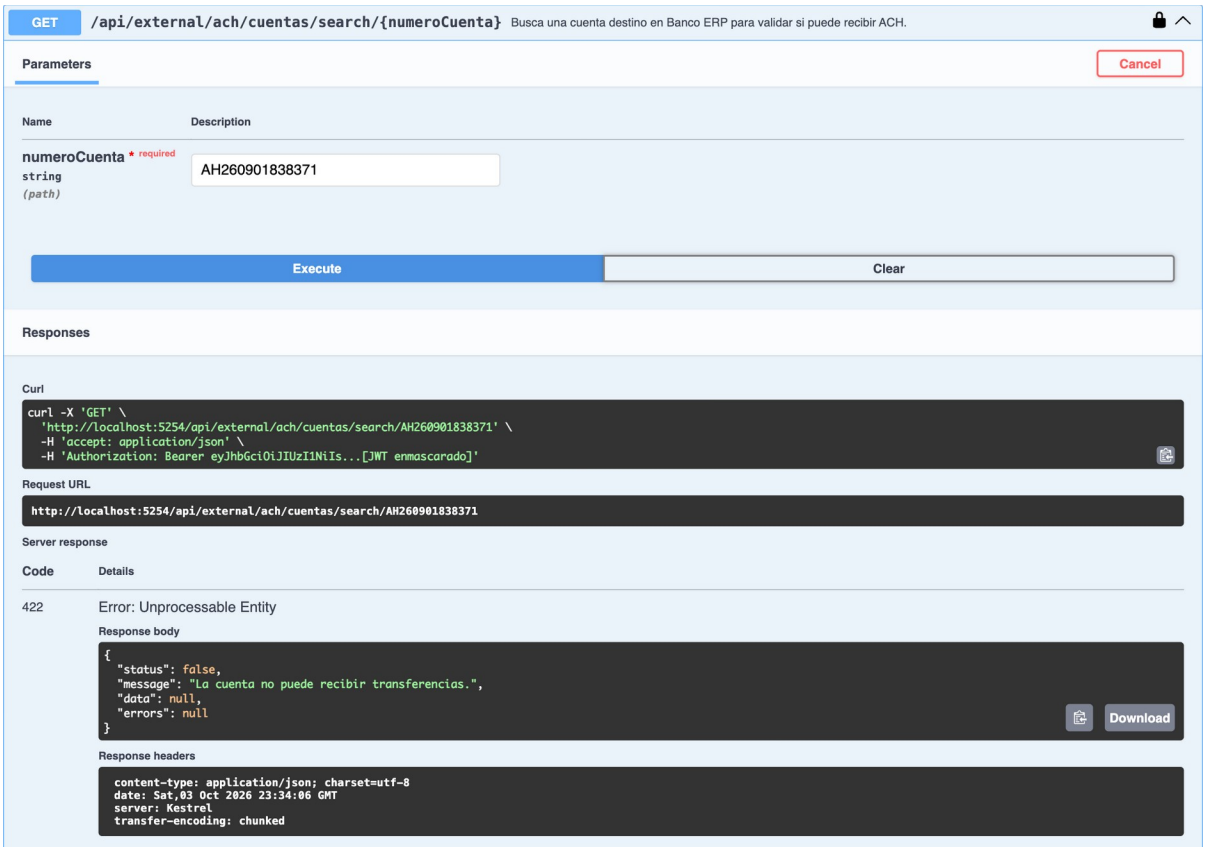


Figura 8. TC-14: cuenta BLOQUEADA, 422.

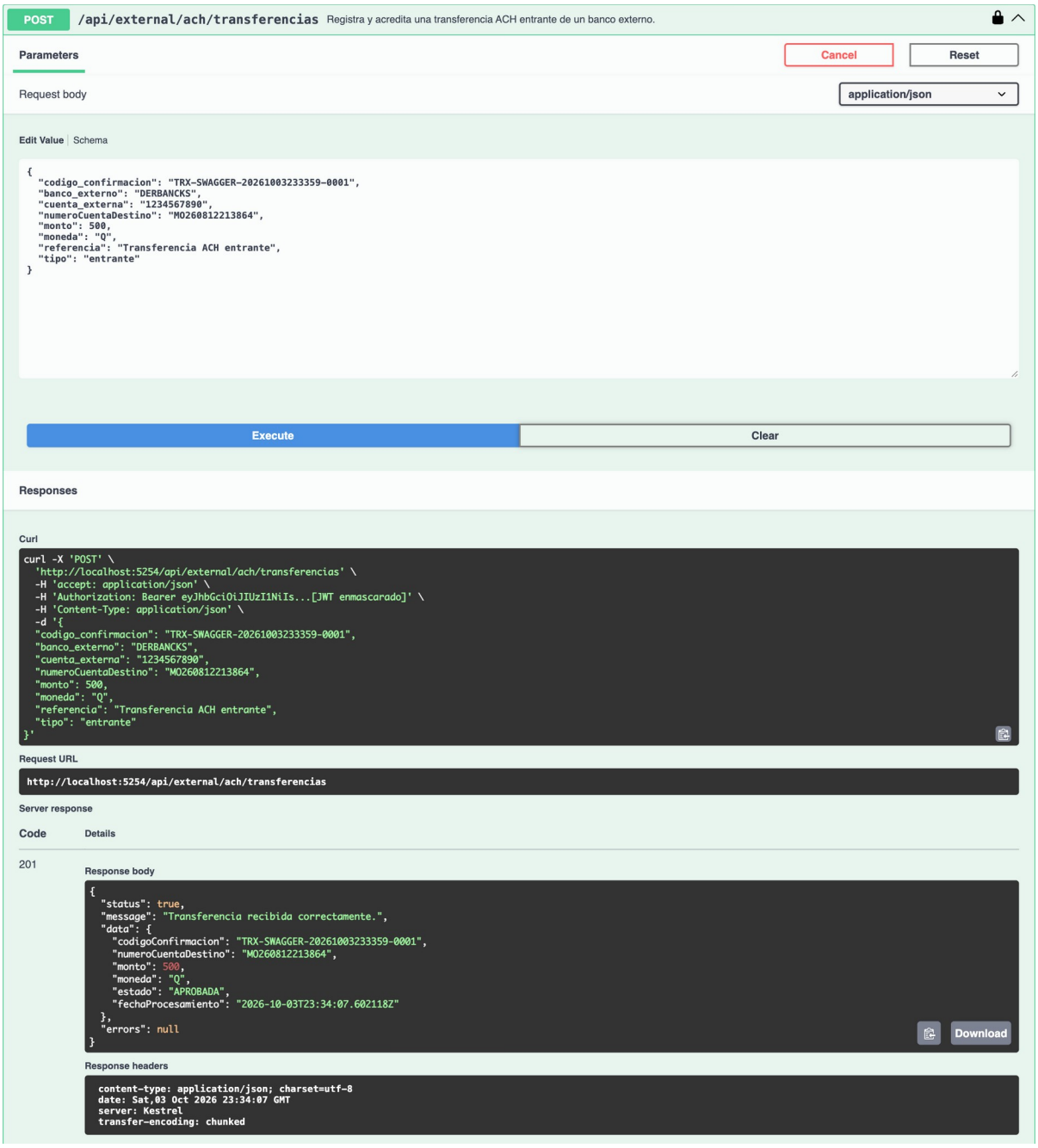


Figura 9. TC-19: transferencia válida, 201 APROBADA (después de las correcciones).

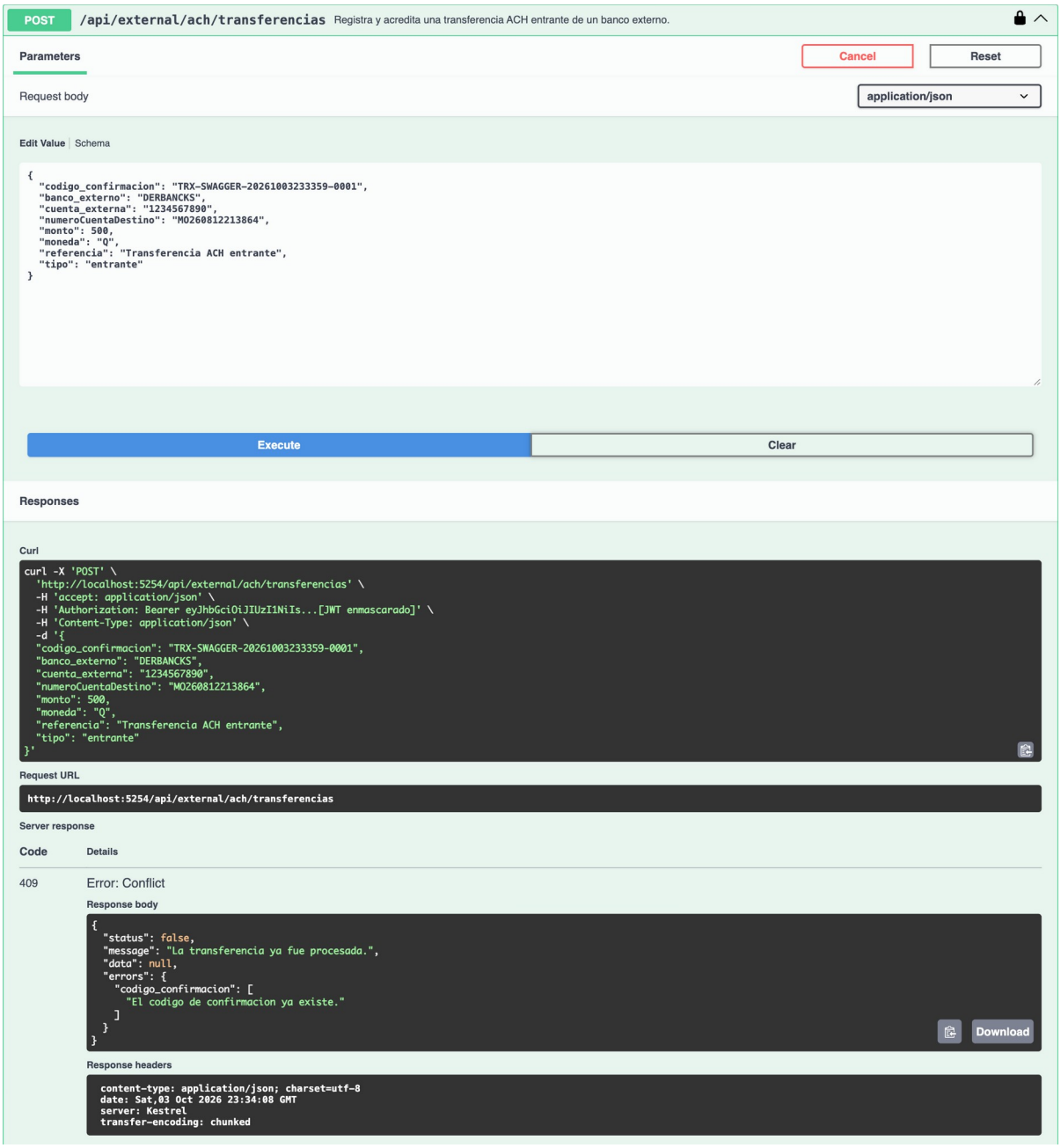


Figura 10. TC-20: mismo codigo_confirmacion, 409.

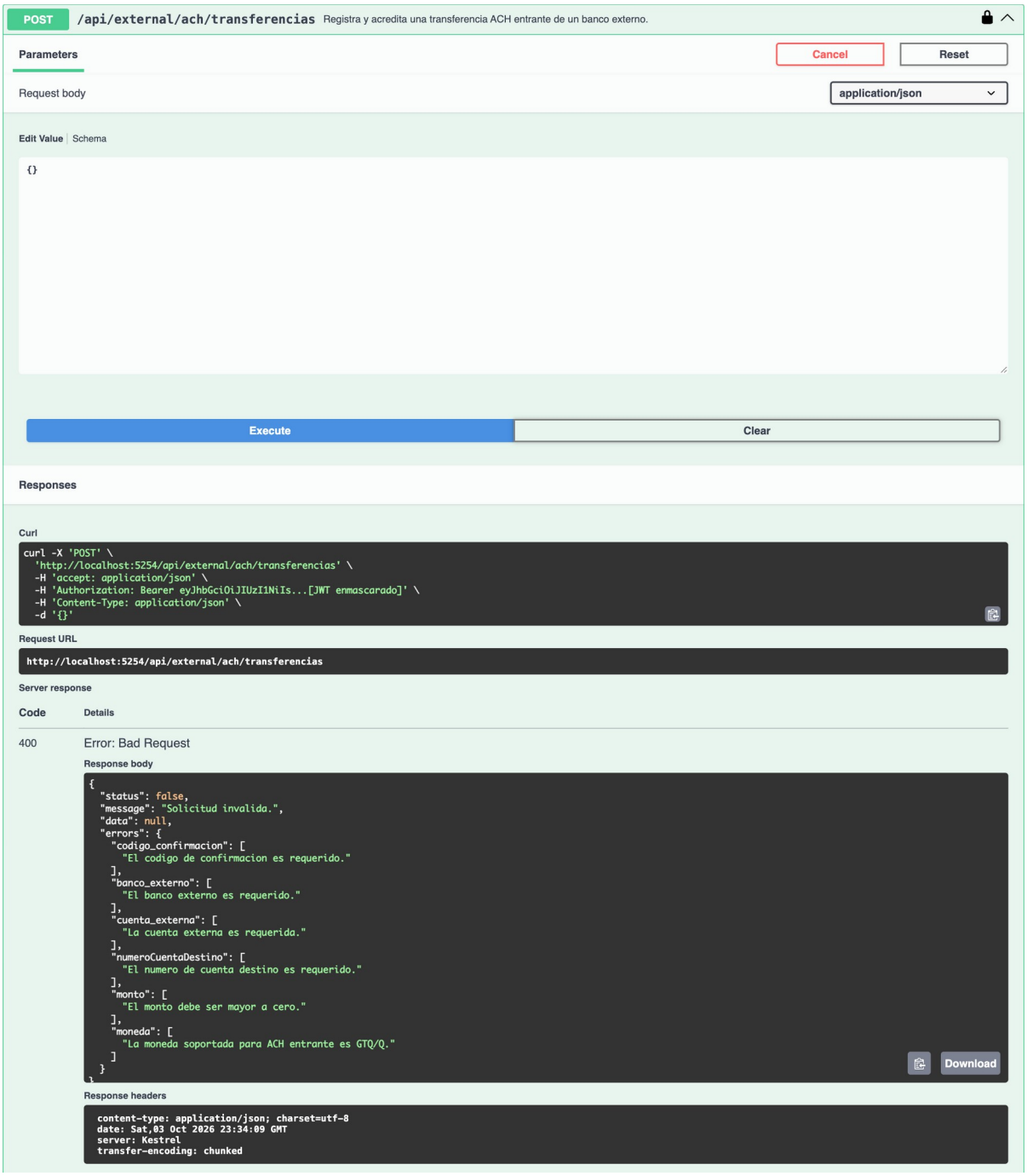


Figura 11. TC-21: body vacío, 400 con errores por campo.

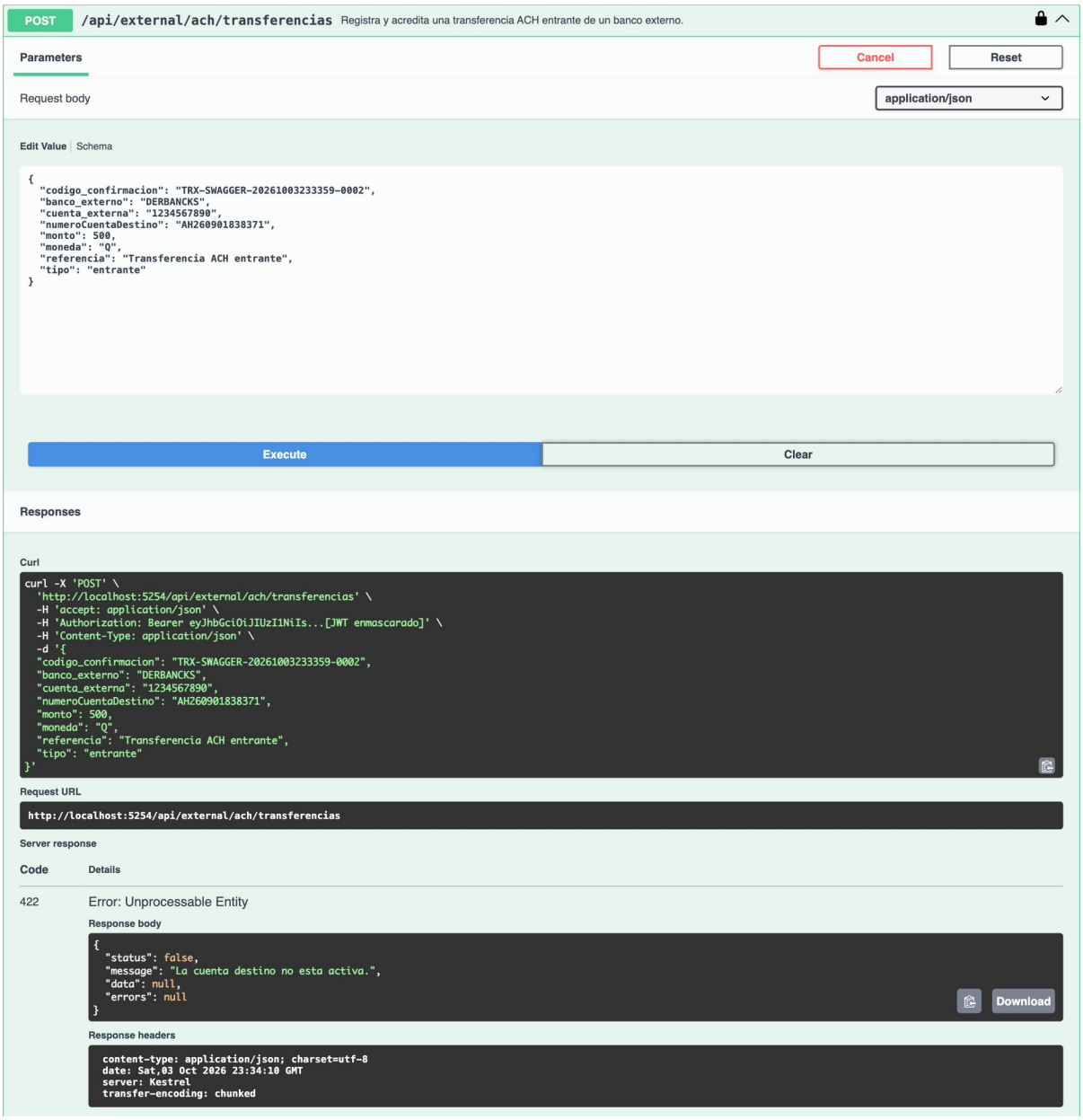


Figura 12. TC-27: transferencia a cuenta bloqueada, 422.

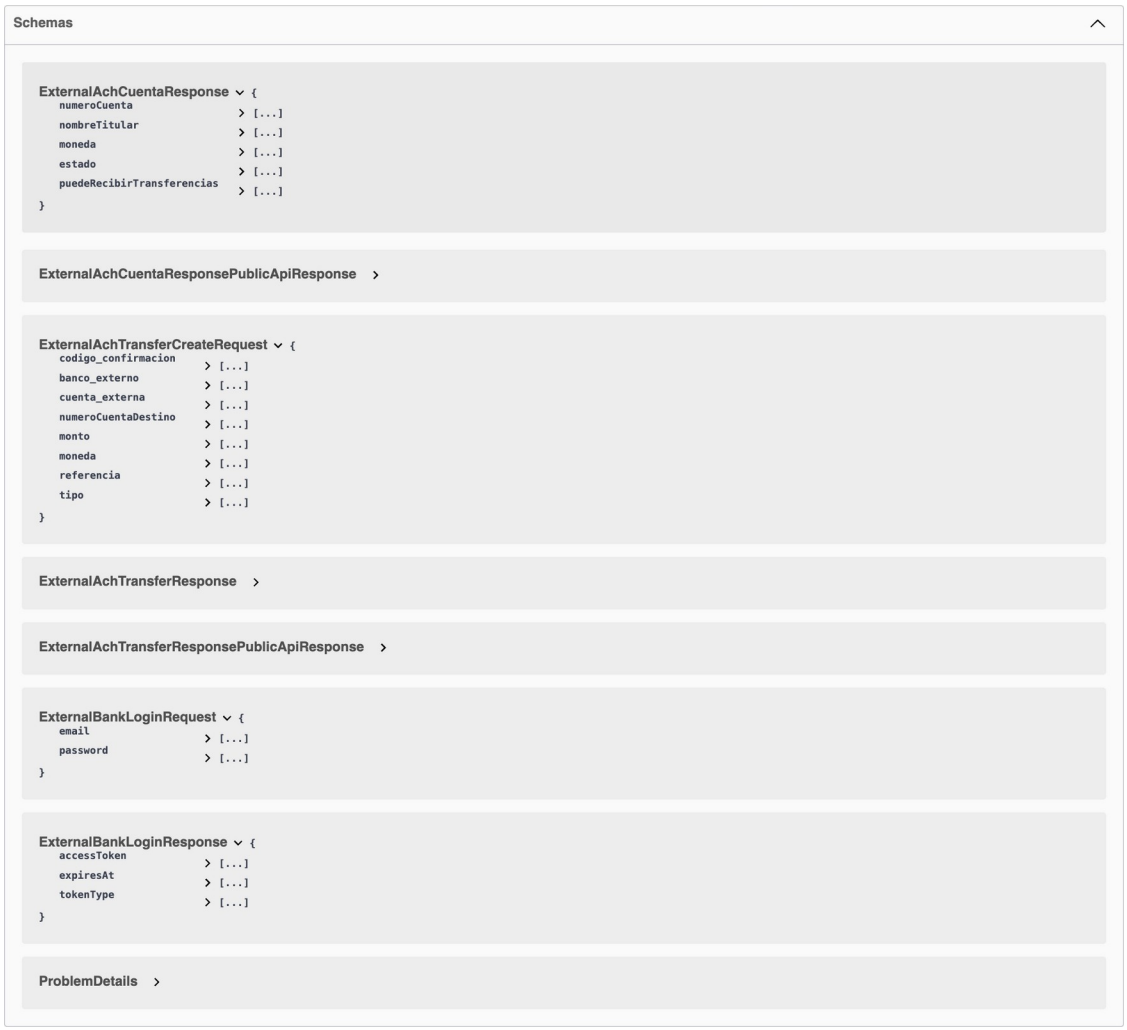


Figura 13. Esquemas OpenAPI de request y response.

D.2 Resultados por caso

Se muestra el resultado de la línea base (C1) y el de la regresión final (C4). El detalle completo de cada ciclo está en cicloN_resultados.json.

ID	Caso	C1	C4 (final)	HTTP (C4)	Tiempo	Evidencia	Observaciones
TC-01	Disponibilidad /api/Home	Pass	Pass	200	34 ms	ciclo4_resultados.json#TC-01	—
TC-02	Login con usuario tecnico valido	Pass	Pass	200	127 ms	ciclo4_resultados.json#TC-02; Fig. 3	—
TC-03	Login con contrasena incorrecta	Pass	Pass	401	92 ms	ciclo4_resultados.json#TC-03; Fig. 4	—
TC-04	Login con usuario inexistente	Pass	Pass	401	3 ms	ciclo4_resultados.json#TC-04	—
TC-05	Login con campos vacios	Pass	Pass	400	1 ms	ciclo4_resultados.json#TC-05	—
TC-06	Login con JSON malformado	Pass	Pass	400	9 ms	ciclo4_resultados.json#TC-06	Pasa, pero responde ProblemDetails de ASP.NET en lugar del contrato {status,message,errors} (DEF-07).
TC-07	Login de usuario inactivo	Pass	Pass	403	2 ms	ciclo4_resultados.json#TC-07	—
TC-08	Login de usuario sin rol BANCO_EXTERNO	Pass	Pass	403	87 ms	ciclo4_resultados.json#TC-08	—
TC-09	Bloqueo tras 5 intentos fallidos	Fail	Pass	423	2 ms	ciclo4_resultados.json#TC-09	C1: falso negativo del script (comparaba "t" contra "true"); se corrigió el script. No es defecto del producto.
TC-10	Login con intento de inyeccion SQL	Pass	Pass	401	3 ms	ciclo4_resultados.json#TC-10	—
TC-11	Consulta de cuenta destino activa	Pass	Pass	200	34 ms	ciclo4_resultados.json#TC-11; Fig. 6	—
TC-12	Consulta de cuenta en minusculas (normalizacion)	Pass	Pass	200	7 ms	ciclo4_resultados.json#TC-12	—
TC-13	Consulta de cuenta inexistente	Pass	Pass	404	7 ms	ciclo4_resultados.json#TC-13; Fig. 7	—
TC-14	Consulta de cuenta BLOQUEADA	Pass	Pass	422	6 ms	ciclo4_resultados.json#TC-14; Fig. 8	—
TC-15	Consulta de cuenta sin token	Pass	Pass	401	1 ms	ciclo4_resultados.json#TC-15; Fig. 5	—
TC-16	Consulta de cuenta con token alterado	Pass	Pass	401	1 ms	ciclo4_resultados.json#TC-16	—
TC-17	Consulta con token de sesion revocada	Pass	Pass	401	1 ms	ciclo4_resultados.json#TC-17	—
TC-18	Consulta con intento de inyeccion SQL	Pass	Pass	404	7 ms	ciclo4_resultados.json#TC-18	—

ID	Caso	C1	C4 (final)	HTTP (C4)	Tiempo	Evidencia	Observaciones
TC-19	Transferencia ACH entrante valida	Fail	Pass	201	22 ms	ciclo4_resultados.json#TC-19; Fig. 9	C1: 422 por DEF-01; C2: 500 por DEF-04. C4: saldo +500.00, ACH_IN, 2 asientos y evento de auditoría verificados.
TC-20	Transferencia duplicada (idempotencia)	Fail	Pass	409	3 ms	ciclo4_resultados.json#TC-20; Fig. 10	C1/C2: no existía transferencia previa (DEF-01/DEF-04); respondía 422/500 en lugar de 409.
TC-21	Transferencia con body vacio	Pass	Pass	400	2 ms	ciclo4_resultados.json#TC-21; Fig. 11	—
TC-22	Transferencia con monto 0	Pass	Pass	400	1 ms	ciclo4_resultados.json#TC-22	—
TC-23	Transferencia con monto negativo	Pass	Pass	400	2 ms	ciclo4_resultados.json#TC-23	—
TC-24	Transferencia con moneda no soportada (USD)	Fail	Fail	400	2 ms	ciclo4_resultados.json#TC-24	Responde 400 "Solicitud invalida."; la guía indica 422 (DEF-06, abierto).
TC-25	Transferencia con tipo distinto de entrante	Pass	Pass	400	1 ms	ciclo4_resultados.json#TC-25	—
TC-26	Transferencia a cuenta inexistente	Fail	Pass	404	13 ms	ciclo4_resultados.json#TC-26	C1: 422 "La cuenta destino no esta activa." (DEF-02).
TC-27	Transferencia a cuenta BLOQUEADA	Pass	Pass	422	8 ms	ciclo4_resultados.json#TC-27; Fig. 12	—
TC-28	Transferencia sin token	Pass	Pass	401	1 ms	ciclo4_resultados.json#TC-28	—
TC-29	Transferencia con monto como texto y moneda GTQ	Fail	Pass	201	11 ms	ciclo4_resultados.json#TC-29	C1/C2: fallaba por DEF-01/DEF-04. C4 verifica moneda "Q" (DEF-08).
TC-30	Transferencia con monto de 3 decimales	Fail	Pass	400	2 ms	ciclo4_resultados.json#TC-30	C1: el monto llegaba al SP sin validar precisión (DEF-05).
TC-31	Transferencia con monto fuera de rango numerico	Fail	Pass	400	2 ms	ciclo4_resultados.json#TC-31	C1: 422 con "numeric field overflow" expuesto (DEF-03, DEF-05).
TC-32	Envio concurrente del mismo codigo (5 solicitudes)	Fail	Pass	201,409,409, 409,409	25 ms	ciclo4_resultados.json#TC-32	C1: 5×422; C2: 5×500; C4: 1×201 + 4×409 y una sola acreditación.
TC-33	Metodo HTTP no soportado	Pass	Pass	405	1 ms	ciclo4_resultados.json#TC-33	—
TC-34	Content-Type no soportado	Pass	Pass	415	2 ms	ciclo4_resultados.json#TC-34	—
TC-35	Tiempo de respuesta de consulta de cuenta (20 llamadas)	Pass	Pass	200	7 ms	ciclo4_resultados.json#TC-35	Promedio 6 ms, p95 7 ms, max 9 ms.

E. Reportar

E.1 Resumen ejecutivo

Indicador	Ciclo 1 (línea base)	Ciclo 4 (final)
Casos ejecutados	35	35
Aprobados (Pass)	26	34
Fallidos (Fail)	9	1
Bloqueados (Blocked)	0	0
Tasa de aprobación	74.3 %	97.1 %
Defectos abiertos	8 detectados	3 abiertos (severidad Baja)

Principales hallazgos:

- **El flujo principal estaba roto: el 100 % de las transferencias válidas fallaba** (DEF-01 y DEF-04). El procedimiento almacenado escribía en columnas inexistentes de contabilidad.asientos y auditoria.log_operaciones, así que todo banco externo que siguiera la guía recibía 422 o 500 y nunca se acreditaban fondos. Está corregido y verificado (TC-19, 20, 29 y 32 en Pass).
- **Fuga de información interna (DEF-03):** el banco externo recibía mensajes de PostgreSQL ("column … does not exist", "numeric field overflow"). Ahora solo se propagan los mensajes de negocio y el resto responde 500 genérico.
- **Validación incompleta del monto (DEF-05):** montos con más de 2 decimales o fuera de rango llegaban a la BD. Ahora se rechazan con 400.
- **Código HTTP incorrecto (DEF-02):** una cuenta destino inexistente respondía 422 "no está activa" en lugar de 404.
- **Fortalezas:** los 11 casos de seguridad de autenticación y autorización pasaron desde la línea base. La idempotencia resiste 5 envíos concurrentes (1×201 + 4×409, una sola acreditación) y el p95 de la consulta fue de 7 ms.

E.2 Incidencias / defectos

ID	Casos	Descripción	Evidencia	Sev.	Recomendación de corrección	Estado
DEF-01	TC-19, 20, 29, 32	ach.sp_registrar_transferencia_entrante_publica inserta en contabilidad.asientos la columna fecha_asiento, que no existe (la tabla usa created_at). Toda transferencia válida se revierte.	ciclo1_resultados.json TC-19: 422 {"message":"column \"fecha_asiento\" of relation \"asientos\" does not exist"}	Alta	Usar created_at en 013 y crear la migración 016 para las BD ya desplegadas. Aplicar 016 en el servidor público antes de habilitar a los bancos.	Corregido; verificado en C3/C4
DEF-02	TC-26	Después de EXECUTE … INTO el SP evalúa IF NOT FOUND, pero EXECUTE no actualiza FOUND. Una cuenta inexistente no se detecta y responde 422 "La cuenta destino no esta activa." en lugar de 404.	ciclo1_resultados.json TC-26	Media	Validar v_cuenta_id IS NULL después del EXECUTE.	Corregido; verificado en C2
DEF-03	TC-19, 31	Cualquier PostgresException se devolvía al cliente como error de negocio (422) con el texto original de la BD: nombres de tablas y columnas, overflow, etc. (CWE-209).	ciclo1_resultados.json TC-31: {"message":"numeric field overflow"}	Alta	Propagar solo RAISE EXCEPTION (SQLSTATE P0001). Otros errores → 500 genérico, con el detalle solo en el log del servidor.	Corregido; verificado en C2
DEF-04	TC-19, 20, 29, 32	Enmascarado por DEF-01: el INSERT en auditoria.log_operaciones usa columnas inexistentes (nombre_usuario, rol_en_momento, ip_address, jti, mensaje_error, duracion_ms, fecha_hora). Las transferencias seguían fallando con 500.	ciclo2_resultados.json TC-19 + log de la API: "42703: column nombre_usuario of relation log_operaciones does not exist"	Alta	Usar el esquema oficial de 002, igual que los SP de 005, y guardar usuario y rol en datos_despues.	Corregido; verificado en C3/C4
DEF-05	TC-30, 31	No se valida la precisión ni el rango del monto: 10.555 llegaba al SP (numeric(18,2) lo redondearía a 10.56 al acreditar) y 1e17 provocaba overflow en la BD.	ciclo1_resultados.json TC-30, TC-31	Media	Validar como máximo 2 decimales y un tope de 9,999,999,999,999,999.99 en ExternalAchPublicValidators, con 3 pruebas unitarias nuevas.	Corregido; verificado en C2
DEF-06	TC-24	Una moneda no soportada (USD) responde 400 "Solicitud invalida.", pero la guía (§8) asigna 422 a "moneda inválida".	ciclo4_resultados.json TC-24	Baja	Definir el contrato: actualizar la guía (400 = validación de formato) o mover la moneda a regla de negocio (422). La prueba unitaria actual fija 400, por lo que no se cambió sin acuerdo del equipo.	Abierto
DEF-07	TC-06	Un JSON malformado responde ProblemDetails de ASP.NET con detalles del parser, no el contrato {status, message, data, errors}. Swagger además documenta ProblemDetails para los errores del login, aunque la API responde PublicApiResponse.	ciclo4_resultados.json TC-06	Baja	Configurar ApiBehaviorOptions.InvalidModelStateResponseFactory y corregir los ProducesResponseType.	Abierto
DEF-08	TC-19, 29	data.moneda de la transferencia devolvía "GTQ", mientras la consulta de cuenta y el ejemplo de Swagger usan "Q". Se detectó al revisar manualmente las evidencias de C3.	ciclo3_resultados.json TC-19	Baja	Normalizar con ToDisplayMoneda en la respuesta y agregar la verificación a TC-19 y TC-29.	Corregido; verificado en C4

Observaciones adicionales (no son defectos funcionales): (1) la guía publica la API en http:// sin TLS, por lo que la contraseña y el JWT viajan en texto plano; (2) Swagger se habilita en todos los ambientes (UseSwagger sin condición), lo que expone el contrato en producción; (3) la cuenta de ejemplo de la guía no coincide con el formato real de las cuentas.

E.3 Conclusiones

Qué cubrió el plan. Los 4 endpoints de la Public API con 35 casos trazados a 12 criterios de aceptación y 8 riesgos, incluidos los tipos de prueba solicitados (funcionales, negativas, seguridad y manejo de errores) más concurrencia y rendimiento básico. Se verificaron los efectos contables en la base de datos y no solo las respuestas HTTP. La regresión final aprobó 34 de 35 casos. La API cumple el flujo de la guía y queda 1 diferencia de contrato abierta (DEF-06).

Qué no se cubrió y por qué. No se probó el servidor público, para no crear transacciones reales ni bloquear la credencial compartida. Además, ese servidor necesita la migración 016 para que las transferencias funcionen. Tampoco se ejecutaron pruebas de carga y estrés, rate limiting, expiración natural del token, TLS ni CORS, porque exceden el alcance y el ambiente local. Las combinaciones parciales de permisos quedaron fuera porque requieren roles personalizados que no existen en los seeds.

E.4 Lecciones aprendidas y mejoras

Lecciones aprendidas:

- Un defecto puede ocultar otro: DEF-01 escondía DEF-04. Hay que repetir la suite completa después de cada corrección, no solo el caso que falló.
- Las verificaciones de caja gris (saldo, ACH_IN, asientos) dan confianza real en una API financiera; un 201 por sí solo no prueba que se movió el dinero.
- La guía, Swagger y el código divergían (DEF-06, 07, 08). El contrato debe tener una única fuente de verdad y probarse de forma automática.
- Los scripts SQL de despliegue no tenían pruebas: el SP fallaba en su primera ejecución real.
- Usar usuarios QA aislados permitió probar el bloqueo sin afectar al usuario técnico real.
- Revisar manualmente las evidencias encontró DEF-08, que ninguna aserción verificaba: las aserciones deben validar el esquema completo de la respuesta.

Cambios propuestos al plan o al proceso:

- Agregar validación de JSON Schema (generado desde OpenAPI) a cada respuesta del runner.
- Ejecutar run_api_tests.sh en el pipeline de CI (docker-compose.ci.yml) con una BD efímera, para que cada cambio de SQL o C# pase por la batería.
- Exigir que todo script nuevo de deploy/production venga acompañado de un smoke test del procedimiento (transacción con ROLLBACK).
- Mantener la guía para bancos versionada junto al código y revisarla en cada PR que toque la Public API.

Siguientes pruebas sugeridas:

- **Regresión en producción:** después de aplicar la migración 016, smoke test controlado con una cuenta de prueba (TC-01, 02, 11, 19 y 20).
- **Carga:** k6 o JMeter con 50 usuarios virtuales en consulta y transferencia (meta p95 < 500 ms y 0 % de errores).
- **Concurrencia sobre la misma cuenta** con códigos distintos, para validar el bloqueo FOR UPDATE y la consistencia del saldo.

- **Seguridad:** escaneo OWASP ZAP baseline, expiración de token, rate limiting en el login y verificación de HTTPS.
- **Contrato:** pruebas de contrato (Schemathesis o Dredd) contra el OpenAPI publicado.

Anexos

Anexo A. Archivos entregados

Archivo	Contenido
scripts/01_preparar_datos_qa.sql	Usuarios QA y reset de bloqueos
scripts/run_api_tests.sh / .mjs	Runner automatizado de los 35 casos
scripts/capturas_swagger.mjs	Capturas automatizadas de Swagger
evidencias/resultados/ciclo1..4_resultados.json	Request, response, tiempos y verificaciones por caso
evidencias/resultados/ciclo1..4_ejecucion.log	Log legible de cada ciclo
evidencias/swagger/01..13_*.png	Capturas de Swagger (Figuras 1–13)

Anexo B. Cambios aplicados al código

Archivo	Cambio	Defecto
Backend/database/deploy/production/ 013_external_public_ach.sql	SP: created_at en asientos, auditoría con el esquema oficial, validación de cuenta inexistente.	DEF-01, 02, 04
Backend/database/deploy/production/ 016_fix_ach_transferencia_entrante.sql	Nueva migración que reemplaza el SP en las BD ya desplegadas.	DEF-01, 02, 04
Backend/database/docker-init/01_bancoerp_init.sh, README_PRODUCTION_DEPLOY.md	Incluyen 016 en el orden de despliegue.	—
Backend/BancoERP.Application/Services/ ExternalAchPublicService.cs	Solo propaga errores P0001; respuesta con moneda "Q".	DEF-03, 08
Backend/BancoERP.Application/Services/ ExternalAchPublicValidators.cs	Valida decimales y rango del monto.	DEF-05
Backend/BancoERP.UnitTests/ExternalPublicApiTests.cs	3 pruebas unitarias nuevas (16/16 aprobadas).	DEF-05

Anexo C. Log de la regresión final (ciclo 4)

```
Ejecucion 2026100323 3357 contra http://localhost:5254 - 2026-10-03T23:33:57.595Z
[PASS] TC-01 Disponibilidad /api/Home -> HTTP 200 (34 ms)
[PASS] TC-02 Login con usuario tecnico valido -> HTTP 200 (127 ms)
[PASS] TC-03 Login con contrasena incorrecta -> HTTP 401 (92 ms)
[PASS] TC-04 Login con usuario inexistente -> HTTP 401 (3 ms)
[PASS] TC-05 Login con campos vacios -> HTTP 400 (1 ms)
[PASS] TC-06 Login con JSON malformado -> HTTP 400 (9 ms)
      obs: Responde con ProblemDetails de ASP.NET (title/errors) en lugar del contrato {status,message,errors}.
[PASS] TC-07 Login de usuario inactivo -> HTTP 403 (2 ms)
[PASS] TC-08 Login de usuario sin rol BANCO_EXTERNO -> HTTP 403 (87 ms)
[PASS] TC-09 Bloqueo tras 5 intentos fallidos -> HTTP 423 (2 ms)
[PASS] TC-10 Login con intento de inyeccion SQL -> HTTP 401 (3 ms)
[PASS] TC-11 Consulta de cuenta destino activa -> HTTP 200 (34 ms)
[PASS] TC-12 Consulta de cuenta en minusculas (normalizacion) -> HTTP 200 (7 ms)
[PASS] TC-13 Consulta de cuenta inexistente -> HTTP 404 (7 ms)
[PASS] TC-14 Consulta de cuenta BLOQUEADA -> HTTP 422 (6 ms)
[PASS] TC-15 Consulta de cuenta sin token -> HTTP 401 (1 ms)
[PASS] TC-16 Consulta de cuenta con token alterado -> HTTP 401 (1 ms)
[PASS] TC-17 Consulta con token de sesion revocada -> HTTP 401 (1 ms)
[PASS] TC-18 Consulta con intento de inyeccion SQL -> HTTP 404 (7 ms)
[PASS] TC-19 Transferencia ACH entrante valida -> HTTP 201 (22 ms)
      obs: Saldo MO260803936604: 31756.91 -> 32256.91.
[PASS] TC-20 Transferencia duplicada (idempotencia) -> HTTP 409 (3 ms)
[PASS] TC-21 Transferencia con body vacio -> HTTP 400 (2 ms)
[PASS] TC-22 Transferencia con monto 0 -> HTTP 400 (1 ms)
[PASS] TC-23 Transferencia con monto negativo -> HTTP 400 (2 ms)
[FAIL] TC-24 Transferencia con moneda no soportada (USD) -> HTTP 400 (2 ms)
      x HTTP 422 (guia seccion 8: moneda invalida) | 400
```

[PASS] TC-25 Transferencia con tipo distinto de entrante -> HTTP 400 (1 ms)
[PASS] TC-26 Transferencia a cuenta inexistente -> HTTP 404 (13 ms)
[PASS] TC-27 Transferencia a cuenta BLOQUEADA -> HTTP 422 (8 ms)
[PASS] TC-28 Transferencia sin token -> HTTP 401 (1 ms)
[PASS] TC-29 Transferencia con monto como texto y moneda GTQ -> HTTP 201 (11 ms)
[PASS] TC-30 Transferencia con monto de 3 decimales -> HTTP 400 (2 ms)
[PASS] TC-31 Transferencia con monto fuera de rango numerico -> HTTP 400 (2 ms)
[PASS] TC-32 Envio concurrente del mismo codigo (5 solicitudes) -> HTTP 201,409,409,409,409 (25 ms)
[PASS] TC-33 Metodo HTTP no soportado -> HTTP 405 (1 ms)
[PASS] TC-34 Content-Type no soportado -> HTTP 415 (2 ms)
[PASS] TC-35 Tiempo de respuesta de consulta de cuenta (20 llamadas) -> HTTP 200 (7 ms)
obs: Promedio 6 ms, p95 7 ms, max 9 ms.

Resumen: 35 ejecutados | 34 Pass | 1 Fail | 0 Blocked